Projekt: Drohnenverwaltung

DI (FH) Markus Falkensteiner

Unterrichtsfach: Systemplanung und Projektentwicklung

Projektauftrag für das Einstiegsprojekt

Requirements · DDD · SDD · OpenSpec · GitHub · KI-Coding-Agent

ASP.NET Core 10 · Angular · Entity Framework Core · SQLite

SYP – 4. INF – 2026/2027

Was euch erwartet

Teil 1 – Das Projekt

Aufgabenstellung, technischer Rahmen und Architektur

Teil 2 – Anforderungen

DDD-Begriffe, Domänenmodell, Domain Rules, REQ-001 bis REQ-010

Teil 3 – Der Workflow

Von OpenSpec über GitHub Issues bis zum getesteten Code

Teil 4 – Abgabe

Change Requests, Traceability, Spielregeln und Bewertung

Ihr arbeitet genau nach dem Workflow, den wir am Beispiel Student Website durchgespielt haben.

Ziel des Projekts

Ein kleines, aber vollständig durchgezogenes Softwareprojekt.

- Anforderungen selbst formulieren, statt sie zu bekommen.
- Eine Fachdomäne mit DDD modellieren und benennen.
- Verhalten mit OpenSpec spezifizieren, bevor Code entsteht.
- Arbeit über GitHub Issues und ein GitHub Project organisieren.
- Einen KI-Coding-Agenten gezielt steuern – Task für Task.
- Jede Codezeile auf eine Anforderung zurückführen können.

Vorbereitung auf das zweite Semester
Dort programmiert ihr eine echte Tello-Drohne – mit Scrum
und demselben technischen Workflow.

Nicht die Menge an Code zählt, sondern die Nachvollziehbarkeit.

1. Das Projekt

Die Aufgabenstellung

Drohnen und Zubehör werden an Schülergruppen ausgeliehen.

- Welche Drohnen gibt es und welche sind gerade verfügbar?
- Welche Gruppe hat welche Drohne ausgeliehen?
- Wie viele Akkus und Propeller wurden ausgegeben?
- Wann wurde ausgeliehen, wann zurückgegeben?
- Welche Ausleihen gab es in der Vergangenheit?

Die Anwendung soll das ersetzen,
was heute ein Zettel an der Schranktür leistet –
nur nachvollziehbar und dauerhaft.

Ihr baut ein Werkzeug, das ihr im zweiten Semester selbst verwenden werdet.

Was das System können muss

Verwalten

Drohnen anlegen und anzeigen.
Schülergruppen anlegen.

Ausleihen

Drohne an eine Gruppe ausgeben,
inklusive Akkus und Propellern.

Zurücknehmen

Ausleihe abschließen und die
Drohne wieder freigeben.

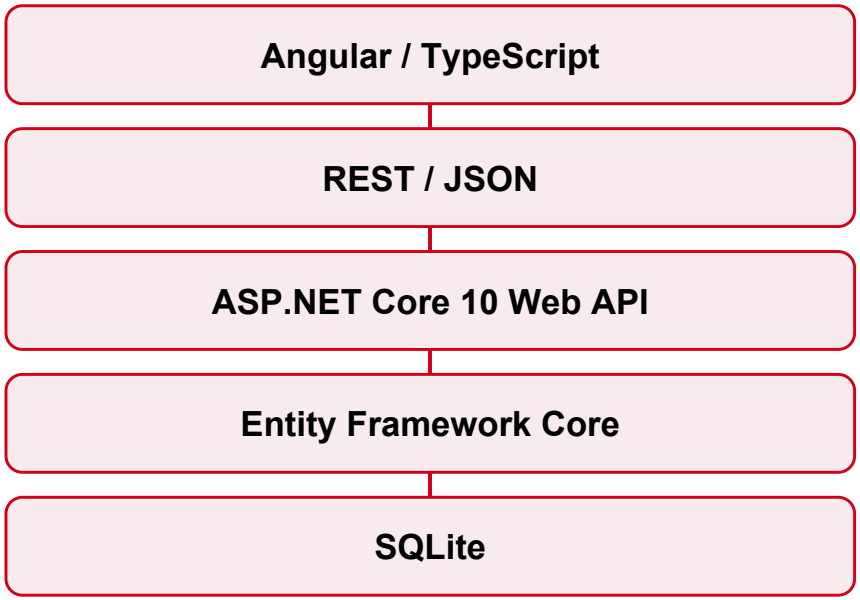
Nachvollziehen

Aktive Ausleihen anzeigen,
alte Ausleihen aufbewahren.

- Der Zustand einer Drohne ist entweder Available oder Loaned.
- Zubehör wird nur als Menge geführt – Akkus werden nicht einzeln inventarisiert.

Eine Drohne darf immer nur eine aktive Ausleihe haben.

Technischer Rahmen



Werkzeuge

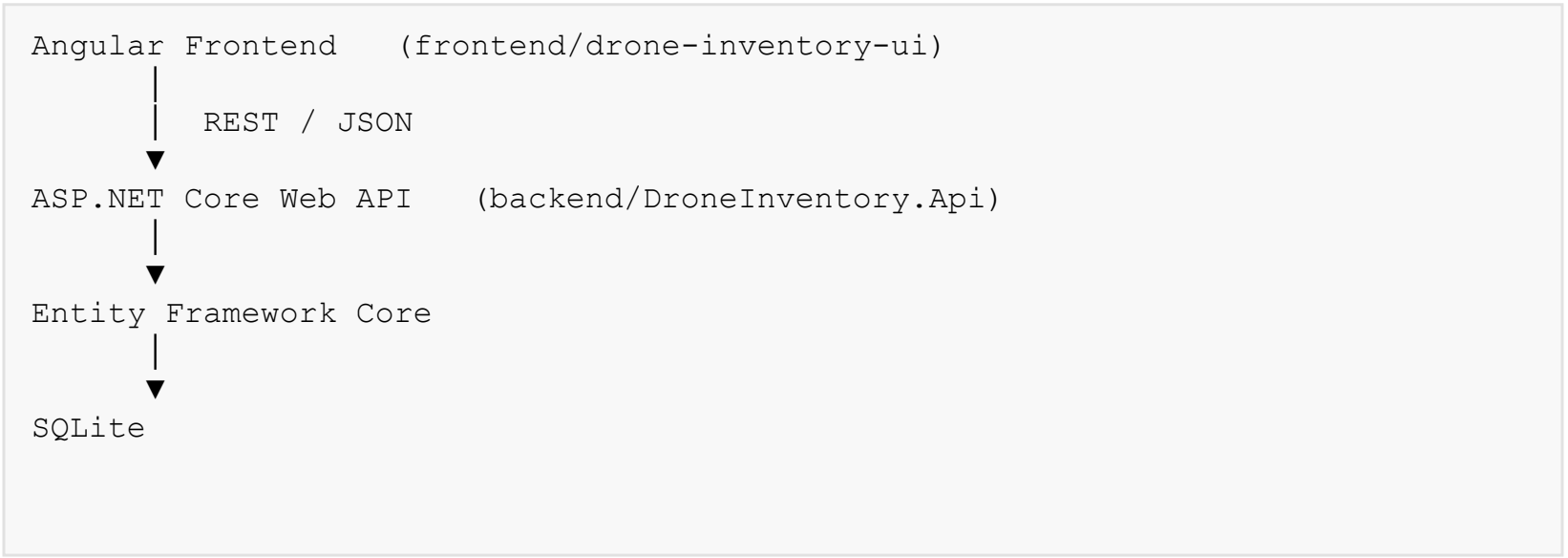
VS Code
Git und GitHub
GitHub CLI (gh)
OpenSpec
KI-Coding-Agent

- Backend in C#, Frontend in Angular – getrennte Projekte in einem Repository.
- Das Backend enthält und erzwingt die fachlichen Regeln.

Die Ausleihregeln gehören ins Backend – nicht in ein Angular-Formular.



Die Architektur



- Das Frontend spricht ausschließlich über die REST-API mit dem Backend.
- Das Frontend greift nie direkt auf die Datenbank zu.
- Geschäftslogik und Validierung liegen im Backend.

Diese Trennung ist eine Vorgabe – sie steht später auch in der config.yaml.



2. Domäne und Anforderungen



DDD: eine gemeinsame Sprache

Drone

Ein Fluggerät, das ausgeliehen werden kann.
Name, Beschreibung, Verfügbarkeit.

StudentGroup

Eine Schülergruppe, die ausleiht
und verantwortlich ist.

Loan

Ein Ausleihvorgang mit Gruppe, Drohne,
Datum und Status.

LoanItem

Eine Zubehörposition einer Ausleihe:
Typ und Menge.

- Verwendet diese Begriffe überall: in Anforderung, Spec, Code, Datenbank und im Gespräch.
- Keine Synonyme wie „Verleih“, „Ausgabe“ oder „Buchung“ erfinden.

Ubiquitous Language: Wenn ihr sauber benennt, versteht auch der Agent die Domäne.



Das Domänenmodell

```
Drone
-----------------
Id
Name
Description
```

```
StudentGroup
-----------------
Id
Name
```

```
Loan
-----------------
Id
DroneId
StudentGroupId
LoanDate
ReturnDate
Status
```

```
LoanItem
-----------------
Id
LoanId
Type (Battery | Propeller)
Quantity
```

- Loan ist die zentrale Entity: sie verbindet Drohne, Gruppe, Zeit und Zubehör.
- LoanItem existiert nur innerhalb einer Ausleihe – ein guter Anlass, über Entity, Value Object und Aggregate zu sprechen.



Domain Rules

- Eine Drohne hat höchstens eine aktive Ausleihe.
- Nur verfügbare Drohnen dürfen ausgeliehen werden.
- Eine Rückgabe macht die Drohne wieder verfügbar.
- Zubehörmengen sind nie negativ.
- Zurückgegebene Ausleihen bleiben gespeichert.
- Akkus und Propeller werden nur als Menge geführt.
- Die Verfügbarkeit ergibt sich aus der aktiven Ausleihe.

Architekturfrage zum Diskutieren

Status als eigenes Datenbankfeld speichern – oder aus der aktiven Ausleihe berechnen? Zwei Wahrheiten sind eine Fehlerquelle.



Die Anforderungen im Überblick

```
REQ-001  Drohne anlegen
REQ-002  Drohnen anzeigen
REQ-003  Schülergruppe anlegen
REQ-004  Drohne ausleihen
REQ-005  Keine doppelte Ausleihe
```

```
REQ-006  Zubehör ausleihen
REQ-007  Aktive Ausleihen anzeigen
REQ-008  Ausleihe zurückgeben
REQ-009  Rückgabedatum speichern
REQ-010  Ausleihhistorie anzeigen
```

- Formuliert jede Anforderung mit SHALL und mindestens einem Given/When/Then-Szenario.
- Die Anforderungen kommen als Dokument ins Repository – sie sind der Input für OpenSpec.
- Zehn Anforderungen sind bewusst wenig – dafür vollständig durchgezogen.

Aus 10 Anforderungen entstehen rund 20 Tasks und ebenso viele Issues.

REQ-001 bis REQ-003 im Detail

```
REQ-001  Drohne anlegen
  Felder: Id, Name, Description (Freitext)
  Beispiel: Name "TELLO-07"
            Description "SSID: TELLO-9FA21B,
                         rechter Propellerschutz beschädigt"

REQ-002  Drohnen anzeigen
  Liste mit Name, Status (Available | Loaned), Beschreibung

REQ-003  Schülergruppe anlegen
  Felder: Id, Name      Beispiel: "Group 3"
```

- Der Status wird nicht eingegeben – er ergibt sich aus der Ausleihsituation.
- Später mögliche Erweiterungen der Gruppe: Class, Members, ResponsibleStudent.

REQ-004 und REQ-005 im Detail

```
REQ-004  Drohne ausleihen
  An available drone SHALL be loaned to a student group.
  Scenario: Loan an available drone
  - GIVEN the drone "TELLO-07" is available
  - AND the student group "Group 3" exists
  - WHEN "TELLO-07" is loaned to "Group 3"
  - THEN an active loan is created

REQ-005  Keine doppelte Ausleihe
  A drone SHALL have at most one active loan.
  - GIVEN "TELLO-07" already has an active loan
  - WHEN another group tries to loan it
  - THEN the loan is rejected
```

REQ-006 und REQ-007 im Detail

```
REQ-006  Zubehör ausleihen
  Erlaubte Typen: Battery, Propeller
  Mengen sind ganzzahlig und >= 0
  Beispiel: Drone TELLO-07, Group 3, 4 Batteries, 8 Propellers

REQ-007  Aktive Ausleihen anzeigen
  Group    | Drone    | Batteries | Propellers | Loan Date
  Group 1  | TELLO-02 |         3 |          4 | 19.08.2026
  Group 3  | TELLO-07 |         4 |          8 | 19.08.2026
```

- Zubehör wird als Menge geführt, nicht als einzelnes Gerät mit eigener Id.
- Negative Mengen müssen vom Backend abgelehnt werden.

REQ-008 bis REQ-010 im Detail

```
REQ-008  Ausleihe zurückgeben
  Loan:  Active   →  Returned
  Drone: Loaned   →  Available
  Scenario: Return a loan
  - GIVEN "TELLO-07" is loaned to "Group 3"
  - WHEN the loan is returned
  - THEN the loan is marked as returned
  - AND "TELLO-07" becomes available again

REQ-009  Rückgabedatum (ReturnDate) speichern
REQ-010  Zurückgegebene Ausleihen bleiben erhalten
```

Zurückgegebene Ausleihen werden nie gelöscht – sonst ist die Historie weg.

Beispiel: eine Ausleihe

```
Student Group:  Group 3
Drone:          TELLO-07
Loan Date:      19.08.2026
Status:         Active

Accessories:    4 Batteries
                8 Propellers
```

- Aus diesem einen Beispiel lassen sich Drone, StudentGroup, Loan und zwei LoanItems ablesen.
- Nach der Rückgabe ändern sich nur Status und ReturnDate – der Datensatz bleibt bestehen.

Prüft eure Specs immer gegen ein konkretes Beispiel.

Bewusst nicht am Beginn

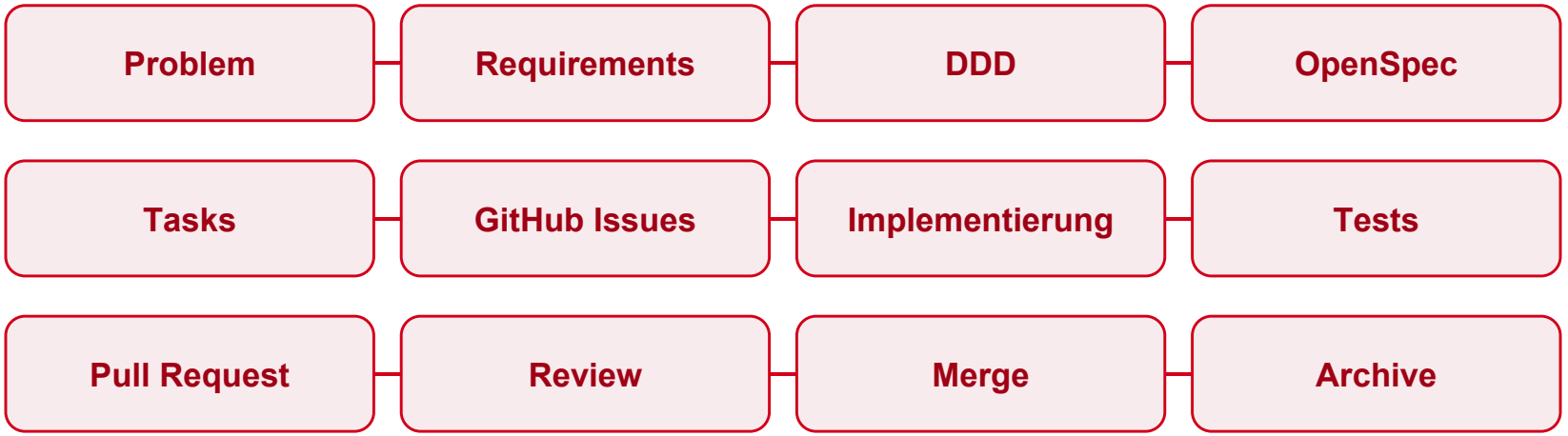
- Kein Login, keine Benutzerkonten, keine Rollen.
- Keine Einzelinventarisierung von Akkus (BAT-001 …).
- Keine Reservierungen im Voraus.
- Keine Schadensverwaltung und keine Rückgabedifferenzen.
- Keine Auswertungen, Statistiken oder Exporte.

Warum diese Grenze wichtig ist
Ein Change soll abschließbar sein. Alles Weitere kommt später
als eigener Change Request – so, wie in echten Projekten.

„Out of Scope“ gehört ausdrücklich in eure Anforderungen und ins proposal.md.

3. Der Workflow

Der Ablauf im Überblick



- Jeder Schritt erzeugt ein Ergebnis, das man lesen und prüfen kann.
- Nach dem Archivieren beginnt der Kreis mit dem nächsten Change von vorne.

Nicht: Prompt → KI → Code. Sondern: verstehen → präzisieren → bauen → prüfen.

Schritt 1: Repository und GitHub Project

```
gh repo create drone-inventory --public --clone
cd drone-inventory
```

- Legt im selben Zug ein GitHub Project für euer Team an.
- Spalten: Backlog, Ready, In Progress, Review, Done.
- Merkt euch den Projektnamen – ihr braucht ihn später in den Skills.

Ein Repository für alles
Backend, Frontend und Specs gehören zusammen –
sonst driften sie auseinander.

Alle Teammitglieder brauchen Schreibrechte auf Repository und Project.



Schritt 2: Backend und Frontend anlegen

```
cd backend
dotnet new webapi -n DroneInventory.Api -f net10.0

cd ../frontend
ng new drone-inventory-ui
```

```
drone-inventory/
├── backend/DroneInventory.Api/
├── frontend/drone-inventory-ui/
├── openspec/
└── README.md
```

- Beide Grundprojekte werden einmal gestartet und committet, bevor es weitergeht.
- Erst wenn das Grundgerüst läuft, beginnt die Spezifikation.

Schritt 3: OpenSpec initialisieren

```
npm install -g @fission-ai/openspec@latest
openspec init
```

```
openspec/
├── specs/     aktueller Sollzustand: Was kann das System?
├── changes/   geplante Änderungen: Was wollen wir ändern?
└── config.yaml
```

- specs/ beschreibt, was das System heute können soll.
- changes/ beschreibt, was sich ändern soll – immer als abgeschlossene Einheit.

Diese Trennung ist der Kern von OpenSpec.

Schritt 4: config.yaml füllen

```
schema: spec-driven

context: |
  Project: Drone Lending System
  Backend:  C#, ASP.NET Core 10 Web API, EF Core, SQLite
  Frontend: Angular, TypeScript
  Domain concepts: Drone, StudentGroup, Loan, LoanItem
  Domain rules:
  - A drone can only have one active loan.
  - Only available drones can be loaned.
  - Returning a loan makes the drone available again.
  - Accessory quantities must not be negative.

rules:
  specs:
    - Use Given/When/Then scenarios for acceptance criteria.
    - Describe behavior, not implementation details.
```

Ein Agent ohne Projektkontext erfindet Annahmen – und zwar überzeugend.

Schritt 5: Arbeitsregeln für den Agenten

```
# AGENTS.md  bzw.  .github/copilot-instructions.md

## Architecture
Angular Frontend  →  REST/JSON  →  ASP.NET Core Web API
                                →  EF Core  →  SQLite

## Rules
- Business logic belongs in the backend.
- The frontend must never access the database directly.
- OpenSpec specifications are the source of truth.

## Project Management
- Use GitHub Issues and the GitHub Project for all work.
```

- Diese Datei wird eingecheckt – der Kontext gehört zum Projekt, nicht in einen Chatverlauf.

Schritt 6: GitHub CLI einrichten

```
gh auth login           # Anmeldung mit Personal Access Token
gh auth refresh -h github.com -s project   # Rechte für Projects
gh project list --owner <OWNER>            # Projektnummer finden
```

- Den Personal Access Token erzeugt ihr in den GitHub-Einstellungen.
- Der Token braucht Rechte für Repository, Issues und Projects.
- Den Token niemals ins Repository committen.

Ohne gh geht es nicht

OpenSpec verwaltet keine Issues. Der Agent verbindet OpenSpec und GitHub über die GitHub CLI.

Schritt 7: Skill openspec-to-github

```
# .github/skills/openspec-to-github/SKILL.md
---
name: openspec-to-github
description: Create and synchronize GitHub Issues from
  OpenSpec requirements and tasks.
---
1. Aktuellen OpenSpec-Change bestimmen.
2. proposal.md, specs/**/*.md und tasks.md lesen.
3. Mit `gh issue list --limit 100` auf Duplikate prüfen.
4. Pro Task ein Issue anlegen, Change-Name in den Body.
5. Issue ins GitHub Project aufnehmen.
6. Issue-Nummer zurück in tasks.md eintragen.
```

- Aufrufbar mit /openspec-to-github – der Agent verwendet ihn bei passender Aufgabe auch selbst.
- Legt zusätzlich openspec-apply-with-github und openspec-archive-with-github an.

Schritt 8: /opsx:propose

```
/opsx:propose initial-drone-lending-system

Create the initial Drone Lending System. The system shall
manage drones, student groups, drone loans, batteries and
propellers as quantities, returns and loan history.
Use the requirements and domain rules from the context.
```

```
openspec/changes/initial-drone-lending-system/
├── proposal.md    Warum? Was ist im Scope?
├── design.md      Technischer Ansatz
├── tasks.md       Arbeitsschritte
└── specs/         drone/, student-group/, loan/
```

Ergebnis sind Dateien – kein Code. Jetzt beginnt die eigentliche Arbeit: lesen.



Schritt 9: Specs reviewen – noch kein Code

- Sind alle zehn Anforderungen abgedeckt?
- Werden unsere Domänenbegriffe verwendet?
- Kann eine Drohne gleichzeitig zweimal ausgeliehen werden?
- Was genau passiert bei einer Rückgabe?
- Darf Zubehör eine negative Menge haben?
- Bleiben zurückgegebene Ausleihen erhalten?
- Sind alle Szenarien testbar formuliert?
- Steht Technik in der Spec, die dort nicht hingehört?

Änderungen zuerst in der Spec – dann implementieren. Eine falsche Spec ergibt einen sauber implementierten Fehler.

Schritt 10: tasks.md prüfen

```
## Backend
- [ ] Drone domain model
- [ ] StudentGroup model
- [ ] Loan domain model
- [ ] Drone API
- [ ] Loan API
- [ ] Return operation
- [ ] Loan validation
- [ ] Backend tests
```

```
## Frontend
- [ ] Drone list
- [ ] Drone form
- [ ] Student group management
- [ ] Loan form
- [ ] Active loan list
- [ ] Return action
- [ ] Loan history
```

- Gute Tasks sind klein, unabhängig und einzeln testbar.
- Ein Requirement ist kein Task: REQ-004 wird zu fünf Arbeitsschritten.

Die Task-Liste ist euer Bauplan für die nächsten Wochen.



Schritt 11: Issues erzeugen und ins Board

```
/openspec-to-github initial-drone-lending-system

# was dabei ausgeführt wird:
gh issue create --title "Implement Loan API" \
  --body "OpenSpec change: initial-drone-lending-system"
gh project item-add 3 --owner <OWNER> \
  --url https://github.com/<OWNER>/drone-inventory/issues/16
```



- Zweimal ausführen darf keine doppelten Issues erzeugen.
- Gearbeitet wird nur an Issues, die auf „In Progress“ stehen.



Schritt 12: Ein Issue umsetzen

```
git switch -c feature/16-loan-api
```

```
/openspec-apply-with-github

Implement GitHub issue #16 according to the corresponding
OpenSpec specification. Only implement this task.
Do not implement other OpenSpec tasks.
```

- Ein Issue = ein Branch = ein Pull Request.
- Der Agent bekommt Spec und Task – nicht „mach das Projekt“.
- Nach dem Lauf: git diff lesen und jede Änderung verstehen.

Häufigster Fehler: dem Agenten das ganze Projekt auf einmal geben.



Schritt 13: Tests, Pull Request, Merge

```
dotnet test
ng test
git diff
git add . && git commit -m "Implement loan API"
git push
```

Pull Request — **Review** — **Merge**

- Jedes Given/When/Then-Szenario braucht einen Test.
- Das Review prüft die Spec-Konformität, nicht nur die Syntax.

„Sieht gut aus“ ist keine Prüfung.

Schritt 14: /opsx:archive

Tasks erledigt — **getestet** — **gemergt** — **/opsx:archive**

```
vorher:   changes/initial-drone-lending-system/specs/loan/spec.md
          specs/loan/spec.md   (noch ohne die neue Regel)
nachher:  specs/loan/spec.md   (Delta eingearbeitet)
          changes/archive/2026-…-initial-drone-lending-system/
```

- Erst archivieren, wenn implementiert, getestet und gemergt ist.
- Danach beschreibt specs/ wieder genau das, was das System kann.

Nicht archivierte Changes bedeuten: die Specs stimmen nicht mehr.

4. Änderungen, Abgabe und Bewertung

Change Requests im Semester

```
1  Beschädigte Drohnen
2  Verantwortlicher Schüler
3  Gruppenmitglieder
4  Einzelne Akkus (BAT-001 …)
5  Fehlendes Zubehör bei Rückgabe
6  Schadensmeldung bei Rückgabe
7  SSID als eigenes Feld
```

Unklare Änderung? Erst /opsx:explore

Ist Battery jetzt eine Entity?
Braucht jeder Akku eine ID?
Kann ein Akku defekt sein?
Gehört ein Akku zu einer Drohne?

- Änderung 4 verändert das Domänenmodell deutlich – aus einer Menge wird eine Entity mit eigener Identität.
- Erst analysieren, dann proposen, dann implementieren.

Die Änderungen bekommt ihr absichtlich erst im Laufe des Projekts – wie in echten Projekten.



So läuft ein Change Request

Neue Anforderung: Beschädigte Drohnen dürfen nicht ausgeliehen werden.

```
/opsx:propose support-damaged-drones

## MODIFIED Requirements
### Requirement: Loan Drone
Only available and undamaged drones SHALL be loaned.

#### Scenario: Damaged drone cannot be loaned
- GIVEN the drone "TELLO-07" is damaged
- WHEN a group tries to loan the drone
- THEN the loan is rejected
```

- Die bestehende Spec wird nicht überschrieben – der Change beschreibt nur das Delta.
- Daraus entstehen neue Tasks, neue Issues, neue Tests.

Genau so sieht kontrolliertes Änderungsmanagement aus.

Traceability: Warum existiert dieser Code?



Bei der Abgabe muss jede Codezeile auf eine Anforderung zurückführbar sein.

Spielregeln für die Arbeit mit der KI

„Code, den du nicht erklären kannst,
gehört nicht in dein Projekt.“

- Verhalten ändern heißt: zuerst die Spec ändern.
- Immer nur einen Task pro Agentenlauf umsetzen lassen.
- git diff vor jedem Commit lesen und verstehen.
- Manuelle Nacharbeit ist erlaubt – aber sie muss sichtbar sein.
- Keine Issues löschen, keine Tasks ungeprüft abhaken.
- Der Agent schlägt vor – die Entscheidung trefft ihr.

Der Prozess ersetzt kein Nachdenken – er macht Nachdenken überprüfbar.

Abgabe und Bewertung

Das wird abgegeben

Repository mit Backend und Frontend
openspec/specs und archivierte Changes
GitHub Issues und gefülltes Project
Pull Requests mit Reviews
Lauffähige Anwendung und Tests

Das wird gefragt

Warum ist Drone eine Entity?
Warum nur eine aktive Ausleihe?
Wo ist diese Regel abgesichert?
Welche Spec fordert das Verhalten?
Welcher Test überprüft es?

- Bewertet wird nicht „die KI hat es geschafft“, sondern Verständnis, Nachvollziehbarkeit und Tests.
- Jedes Teammitglied muss den gesamten Workflow erklären können – nicht nur den eigenen Teil.

Vollständigkeit der Spur zählt mehr als die Anzahl der Features.

Euer Fahrplan

1. Anforderungen erstellen
2. Domäne und Begriffe klären
3. DDD-Modell skizzieren
4. Repository und Project anlegen
5. Backend und Frontend anlegen
6. openspec init
7. config.yaml und AGENTS.md füllen
8. GitHub CLI einrichten
9. Skills anlegen
10. /opsx:propose

11. Specs gemeinsam reviewen
12. tasks.md prüfen
13. Issues erzeugen, Board füllen
14. Issue implementieren
15. Tests ausführen
16. Pull Request und Review
17. Merge
18. /opsx:archive
19. Change Request → zurück zu 10
20. Abgabe und Präsentation

Ab Schritt 19 dreht sich der Kreis – genau das ist der Unterschied zum Wasserfall.

Unterlagen und Quellen

Beispielprojekt Student Website

github.com/falkm1980/openSpecStudentWebsite – derselbe Workflow, durchgespielt mit Python, FastAPI und SQLite.

OpenSpec

github.com/Fission-AI/OpenSpec – Dokumentation: docs/concepts.md und docs/getting-started.md

Werkzeuge

openspec.dev · cli.github.com · GitHub Projects
Domain-Driven Design: Eric Evans (2003)

Der Projektauftrag im Detail steht im SYP-Leitfaden „Drohnenverwaltung“.